

Projeto e Análise de Algoritmos I

Aula 03 - Correção de Algoritmos e Invariantes

Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica



Estes slides utilizam conteúdo adaptado da bibliografia da disciplina e também de notas de aula e slides prévios dos professores Bruno Grisci, Rodrigo Machado e André Grahl.

Programa

- Correção de Algoritmos
- Problema da Ordenação de Vetores
- Princípio da Invariante
- Algoritmos Iterativos de Ordenação
- Algoritmos Recursivos de Ordenação

Correção de Algoritmos

“

Uma demonstração de correção convincente será impossível enquanto o algoritmo seja visto como uma caixa preta, nossa única esperança é não considerar o algoritmo como uma caixa preta.

— Edsger W. Dijkstra

“Notes On Structured Programming”, 1970

Introdução

Def. Um **algoritmo** é qualquer *procedimento computacional bem definido* que recebe um conjunto de valores como **entrada** e produz um conjunto de valores como **saída**.

Obs. Um algoritmo pode ser visto como uma ferramenta para resolver um problema.

Def. Um algoritmo **resolve um problema** se para toda entrada:

- Ele **pára**.
- Retorna a saída **correta**.

P. Um algoritmo que não pára é útil? Um algoritmo incorreto é útil?

Problemas e Instâncias

Problema. Especificação **abstrata e genérica** da relação entre um conjunto de entradas válidas e as saídas desejadas.

Instância. Um **caso concreto** ou entrada específica (com dados fixos) que satisfaz as restrições do problema.

Exemplo de Problema: Ordenação de Vetores

- **Entrada:** Sequência de n números $\langle a_1, a_2, \dots, a_n \rangle$.
- **Objetivo:** Permutação $\langle a'_1, a'_2, \dots, a'_n \rangle$ tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Exemplo de Instância: Uma entrada específica

- **Entrada:** $\langle 31, 41, 59, 26, 41, 58 \rangle$
- **Solução:** $\langle 26, 31, 41, 41, 58, 59 \rangle$

Obs. Um algoritmo correto deve produzir a solução esperada para **todas** as

Correção de Algoritmos

- Como discutido, um *algoritmo* é uma sequência de passos para a resolução de um *problema* ou *cálculo* de uma quantidade.
- Um *algoritmo* está **correto** quando a sua respectiva saída corresponde à solução ou quantidade esperada para **todas as possíveis entradas**.
- Exemplo de **algoritmo (incorreto)** para calcular números primos

$$f(n) = n^2 + n + 41$$

(gera números primos somente enquanto $n < 40$)

- Antes de pensarmos no *custo do algoritmo*, é importante confirmar que ele *realmente* resolve o problema!

```
Python Code ↺ Start Over ▶ Run Code  
1 def f(n):  
2     return n**2 + n + 41  
3  
4 f(1), f(2), f(39), f(40) # 1681 é divisível por 41
```

Testando a Correção de Algoritmos

- Determinar a **correção** de algoritmos/programas pode depender muito da descrição do mesmo e de seu estilo.
- Uma primeira forma de aferir correção é **testar** o algoritmo para diversas entradas.
- Contudo, testes podem aferir a **presença de erros**, mas não a **ausência** dos mesmos (a menos que cubram todas as possíveis entradas - o que normalmente é impossível por questão de infinitude)
- Dessa forma, **provas** de correção são necessárias para garantir que algoritmos estejam corretos em relação ao problema que visam resolver.

Provando a Correção de Algoritmos

Algumas técnicas importantes para provar a correção de algoritmos:

- Manipulação de identidades matemáticas e algébricas
- Técnicas de dedução da **lógica formal** (dedução direta, redução ao absurdo, contraposição, etc.)
- Técnicas envolvendo **recursão** e **indução matemática**
- Determinação de pré-condições, pós-condições, e aferição de **invariantes de laços**

Além da resposta do algoritmo estar correta, é também necessário estipular a **garantia de terminação** do algoritmo (i.e. variantes para laços)

Correção dos Algoritmos Vistos Previamente

Problema do Troco (Greedy Cashier's Algorithm):

- Correção baseada em propriedades matemáticas do conjunto de moedas usados para fornecer o troco.

Emparelhamento Estável (Gale-Shapley):

- Correção baseada em um raciocínio por contradição (se a resposta do algoritmo não for um emparelhamento estável, é possível deduzir uma contradição lógica)

Problema da Ordenação de Vetores

Ordenação de Vetores

Entrada. Uma sequência de k números $\langle a_1, a_2, \dots, a_k \rangle$.

Objetivo. Encontrar uma permutação $\langle a'_1, a'_2, \dots, a'_k \rangle$ em que $a'_1 \leq a'_2 \leq \dots \leq a'_k$.

Instância: $\langle 31, 41, 59, 26, 41, 58 \rangle$

- Satisfaz todas as restrições impostas pela definição do problema.

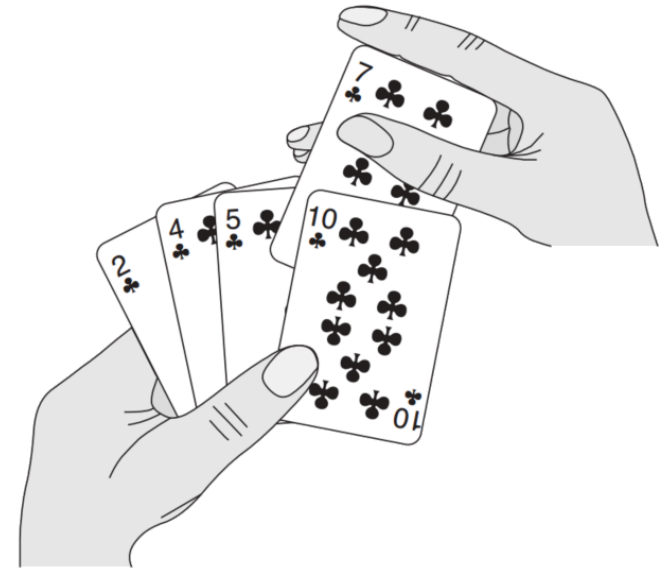
Solução. $\langle 26, 31, 41, 41, 58, 59 \rangle$.

Algoritmos Iterativos de Ordenação

Ordenação por Inserção

Funciona da forma que um humano organiza uma mão jogando cartas:

- Inicia com a mão da esquerda vazia e com as cartas viradas para baixo na mesa.
- Remove uma carta de cada vez e insere na posição correta na mão esquerda.
- Para encontrar a posição correta da carta, compara com as cartas que já estão na mão esquerda.
- **Invariante:** Em todo momento todas as cartas na mão esquerda estão ordenadas.



Ordenação por Inserção (InsertionSort)

```
INSERTIONSORT( $A$ )
1:   $n \leftarrow \text{length}(A)$ 
2:  for  $j \leftarrow 2$  to  $n$  do
3:       $\text{key} \leftarrow A[j]$ 
4:       $i \leftarrow j - 1$ 
5:      while  $i > 0$  and  $A[i] > \text{key}$  do
6:           $A[i + 1] \leftarrow A[i]$ 
7:           $i \leftarrow i - 1$ 
8:       $A[i + 1] \leftarrow \text{key}$ 
```

Obs. Nos pseudocódigos de todos os algoritmos desta aula, assumimos indexação de 1 a n (para vetores de tamanho n).

Princípio da Invariante

Corretude

Def. Uma execução de um algoritmo é uma **sequência** de estados com a propriedade de que:

- Tem um estado inicial; e
- Se q e r são estados consecutivos na **sequência**, então $q \longrightarrow r$.

Def. Um estado é **alcançável** se aparece em alguma execução.

Def. Uma **invariante** é uma propriedade P para estados:

- Tal que se $P(q)$, o estado q possui a propriedade P ; e
- $q \longrightarrow r$; então
- $P(r)$, o estado r possui a propriedade P .

Princípio da Invariante

Def. Uma **invariante** é uma propriedade P para estados:

- Tal que se $P(q)$, o estado q possui a propriedade P ; e
- $q \longrightarrow r$; então
- $P(r)$, o estado r possui a propriedade P .

Princípio da Invariante. Se o estado inicial possui a invariante, então **todos os estados alcançáveis possuem a invariante**.

Princípio da Invariante

Item 1. Define a propriedade.

Item 2. Prova que a propriedade se mantém para $q \longrightarrow r$.

Item 3. Prova que a propriedade se mantém para qualquer estado alcançável.

- Prova por indução.
- Caso Base.
- Passo de Indução.
- Assume $P(n)$ prova $P(n + 1)$.

Item 4. Prova que o algoritmo termina.

Item 5. Conclui que o algoritmo resolve o problema.

Busca Linear - Invariante

BUSCA-LINEAR(A, n, x)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

Invariante do Laço. No início de cada iteração do laço For, se x está presente no vetor A , então x está presente no subvetor $A[i \dots n]$.

Terminação e Corretude.

- Se $A[i] = x$: O algoritmo retorna o índice i imediatamente (correto).
- Se $i = n + 1$ (fim do laço): Considere a **contrapositiva** do invariante:
- “Se x não está em $A[i \dots n] \implies x$ não está em A ”
- Para $i = n + 1$, o subvetor $A[n + 1 \dots n]$ é **vazio**.
- Como x não pode estar em um subvetor vazio $\implies x$ não está em A .
- Logo, retornar **Não-Encontrado** está correto. ■

Invariantes: Algoritmos de Ordenação Iterativos

Corretude: InsertionSort

Def. *Ordenado*(q): A subsequência representada pelo estado q de $j - 1$ números $\langle a'_1, a'_2, \dots, a'_{j-1} \rangle$ obedece a restrição de que $a'_1 \leq a'_2 \leq \dots \leq a'_{j-1}$.

Lema 1. Para qualquer transição $q \longrightarrow r$ do algoritmo de Ordenação por Inserção, se *Ordenado*(q) então *Ordenado*(r).

Prova. A demonstração do lema segue da construção do loop while.

Durante a execução do loop, o algoritmo move para a direita $A[j - 1]$, $A[j - 2]$, $A[j - 3]$, e assim por diante, até que a posição correta de *key* é encontrada.

Nesse ponto *key* é colocada na posição correta e a subsequência está ordenada.



InsertionSort: Terminação

Lema 2. O algoritmo de Ordenação por Inserção termina.

Prova. O loop externo termina quando $j > n$, isso ocorre quando $j > A.length$ e j é apenas incrementado durante o algoritmo.



Corolário. O algoritmo de Ordenação por Inserção resolve o problema de ordenação.

Prova. Pelo **Lema 2**, sabemos que o algoritmo termina depois de n iterações. Pelo **Lema 1** sabemos que toda subsequência alcançada pelo algoritmo está ordenada, como na iteração $n + 1$ a subsequência corresponde a todo o vetor, então podemos concluir que o algoritmo resolve o problema.



Princípio da Invariante

Item 1. Define a propriedade.

Item 2. Prova que a propriedade se mantém para $q \longrightarrow r$.

Item 3. Prova que a propriedade se mantém para qualquer estado alcançável.

- Prova por indução.
- Caso Base.
- Passo de Indução.
- Assume $P(n)$ prova $P(n + 1)$.

Item 4. Prova que o algoritmo termina.

Item 5. Conclui que o algoritmo resolve o problema.

Princípio da Invariante: Criador



- Formulado por Robert W. Floyd na Carnegie-Mellon University em 1967.
- Jovem prodígio que nunca concluiu o doutorado.
- Princípio simples e amplamente aplicável.
- Professor em Stanford e Prêmio Turing em 1978.
- Robert W Floyd, por Donald E. Knuth:
<http://oldwww.acm.org/pubs/membernet/stories/floyd.p>

Exercício 1: Soma dos Elementos

Qual é o invariante do laço no algoritmo a seguir?

```
SOMA( $A, n$ )  
1:   $s \leftarrow 0$   
2:  for  $i \leftarrow 1$  to  $n$  do  
3:     $s \leftarrow s + A[i]$   
4:  return  $s$ 
```

Exercício. Determine a propriedade invariante do laço For e o argumento de terminação.

Exercício 1: Solução

Solução.

- **Invariante do laço:** No início de cada iteração i ($1 \leq i \leq n + 1$), a variável s contém a soma dos elementos da subsequência $A[1 \dots i - 1]$:

$$s = \sum_{k=1}^{i-1} A[k]$$

- **Base ($i = 1$):** $s = 0$ (soma de conjunto vazio).
- **Passo ($i \rightarrow i + 1$):** adiciona-se $A[i]$ a s , mantendo $s = \sum_{k=1}^i A[k]$.
- **Terminação:** O laço termina quando $i = n + 1$. Pelo invariante, $s = \sum_{k=1}^n A[k]$, ou seja, a soma de todo o vetor A . ■

Exercício 2: Maior Elemento

Qual é o invariante do laço no algoritmo a seguir?

MAIORELEMENTO(A, n)

```
1:   $m \leftarrow A[1]$ 
2:  for  $i \leftarrow 2$  to  $n$  do
3:      if  $A[i] > m$  then
4:           $m \leftarrow A[i]$ 
5:  return  $m$ 
```

Exercício. Determine a propriedade invariante do laço For e o argumento de terminação.

Exercício 2: Solução

Solução.

- **Invariante do laço:** No início de cada iteração i ($2 \leq i \leq n + 1$), a variável m armazena o maior valor presente no subvetor $A[1 \dots i - 1]$:

$$m = \max(A[1 \dots i - 1])$$

- **Base ($i = 2$):** $m = A[1] = \max(A[1 \dots 1])$.
- **Passo ($i \rightarrow i + 1$):** se $A[i] > m$, m é atualizado para $A[i]$, mantendo $m = \max(A[1 \dots i])$.
- **Terminação:** O laço termina quando $i = n + 1$. Pelo invariante, $m = \max(A[1 \dots n])$, garantindo que m é o maior elemento de todo o vetor A . ■

Ordenação por Seleção (SelectionSort)

SELECTIONSORT(A)

```
1:   $n \leftarrow \text{length}(A)$ 
2:  for  $i \leftarrow 1$  to  $n$  do
3:       $m \leftarrow i$ 
4:      for  $j \leftarrow i$  to  $n$  do
5:          if  $A[j] < A[m]$  then
6:               $m \leftarrow j$ 
7:      Swap( $A, i, m$ )
```

SWAP(A, x, y)

```
1:   $tmp \leftarrow A[x]$ 
2:   $A[x] \leftarrow A[y]$ 
3:   $A[y] \leftarrow tmp$ 
```

SelectionSort: Correção e Terminação

Qual a intuição por trás da construção do **SelectionSort**?

Determine:

1. O invariante do corpo do laço
2. O respectivo argumento de terminação

SelectionSort: Correção e Terminação (Resolução)

- **Invariante do laço:** No início de cada iteração i (do laço externo), o subvetor $A[1 \dots i - 1]$ contém os $i - 1$ menores elementos do vetor original, e encontra-se **ordenado**.
- **Terminação:** O laço externo termina após $i = n$. Neste ponto, o subvetor $A[1 \dots n]$ conterá os n menores elementos, significando que logo todo o vetor A está **ordenado**. ■

BubbleSort

```
BUBBLESORT( $A$ )
1:   $n \leftarrow \text{length}(A)$ 
2:  for  $i \leftarrow 1$  to  $n - 1$  do
3:      for  $j \leftarrow 1$  to  $n - i$  do
4:          if  $A[j] > A[j + 1]$  then
5:              Swap( $A, j, j + 1$ )
```

BubbleSort: Correção e Terminação

Qual a intuição por trás da construção do **BubbleSort**?

Determine:

1. O invariante do corpo do laço
2. O respectivo argumento de terminação

BubbleSort: Correção e Terminação (Resolução)

- **Invariante do laço:** No início da iteração i do laço externo ($1 \leq i \leq n - 1$), os $i - 1$ maiores elementos do vetor original já estão em suas posições finais, ocupando exatamente as posições

$$A[n - (i - 1) + 1], A[n - (i - 1) + 2], \dots, A[n],$$

isto é, o subvetor

$$A[n - i + 2 \dots n],$$

e esses elementos estão **ordenados**.

- **Terminação:** O laço externo executa para $i = 1, 2, \dots, n - 1$. Quando ele termina, os $n - 1$ maiores elementos estão em suas posições finais ($A[2], \dots, A[n]$). O único elemento restante ocupa $A[1]$, portanto todo o vetor A está **ordenado**. ■

Algoritmos Recursivos de Ordenação

MergeSort

- **MergeSort** é um dos mais famosos e eficientes algoritmos de ordenação baseado em comparações.
- A sua ideia fundamental é uma rotina que une **duas listas ordenadas** em uma **lista maior ordenada** (merge).
- O algoritmo divide a entrada ao meio até chegar ao caso base (vetor de tamanho 1 - trivialmente ordenado), e após recombina os vetores ordenados.

MergeSort: Ideia Principal

MERGE_SORT(A)

```
1:  if length( $A$ ) < 2 then
2:      return  $A$ 
3:  else
4:       $r_1 \leftarrow \text{MergeSort}(\text{FirstHalf}(A))$ 
5:       $r_2 \leftarrow \text{MergeSort}(\text{SecondHalf}(A))$ 
6:      return Merge( $r_1, r_2$ )
```

- No pseudocódigo acima, as rotinas **FirstHalf** e **SecondHalf** extraem a primeira e segunda metade do vetor, respectivamente, e **Merge** une dois vetores ordenados em um único vetor ordenado.
- Implementações eficientes de **MergeSort** requerem o uso de um vetor auxiliar e manipulação de índices (para que os cortes tenham custo constante).

MergeSort: Main (Eficiente)

MERGE_SORT($A, start, end$)

- 1: **if** $start < end$ **then**
 - 2: $mid \leftarrow \lfloor (start + end) / 2 \rfloor$
 - 3: MergeSort($A, start, mid$)
 - 4: MergeSort($A, mid + 1, end$)
 - 5: Merge($A, start, mid, end$)
-

Obs. $start$ e end representam os índices de início e fim da parte do vetor a ser ordenado. Para ordenar o vetor inteiro, chame MergeSort($A, 1, \text{length}(A)$).

MergeSort: Merge (Eficiente)

MERGE($A, start, mid, end$)

```
1:  for  $i \leftarrow start$  to  $end$  do
2:     $Aux[i] \leftarrow A[i]$ 
3:   $i \leftarrow start$ 
4:   $j \leftarrow mid + 1$ 
5:   $k \leftarrow start$ 
6:  while  $i \leq mid$  and  $j \leq end$  do
7:    if  $Aux[i] < Aux[j]$  then
8:       $A[k] \leftarrow Aux[i]$ 
9:       $i \leftarrow i + 1$ 
10:   else
11:      $A[k] \leftarrow Aux[j]$ 
12:      $j \leftarrow j + 1$ 
13:    $k \leftarrow k + 1$ 
14:  while  $i \leq mid$  do
15:     $A[k] \leftarrow Aux[i]$ 
16:     $i \leftarrow i + 1$ 
17:     $k \leftarrow k + 1$ 
18:  while  $j \leq end$  do
19:     $A[k] \leftarrow Aux[j]$ 
20:     $j \leftarrow j + 1$ 
21:     $k \leftarrow k + 1$ 
```

- Há um vetor auxiliar Aux que recebe a porção do vetor a ser ordenada (linhas 1-2), contendo os dois subvetores já ordenados (de $start$ a mid , e de $mid + 1$ a end)
- i aponta para o início do primeiro subvetor ordenado em Aux (inicia com $start$)
- j aponta para o início do segundo subvetor ordenado em Aux (inicia com $mid + 1$)
- k aponta para a posição onde os elementos dos dois subvetores serão inseridos em A (inicia com $start$)
- o laço das linhas 6-12 compara o menor de cada subvetor, inserindo na posição correta do vetor de saída (e atualizando os índices)
- o laço das linhas 13-16 e 17-20 copiam os elementos de um dos subvetores quando o outro subvetor já terminou.

MergeSort: Correção e Terminação

Para determinar a correção do **MergeSort** utilizamos **indução estrutural em sequências de números**.

- determinar que o algoritmo é correto para os casos-base (onde a recursão não é chamada)
- assumindo que chamadas do algoritmo sobre **sequências menores** que a atual provêm a resposta correta, provar que a saída calculada para a sequência atual é **correta**.

QuickSort

QuickSort é outro algoritmo famoso e bastante utilizado.

- **Pivô:** escolha de um elemento arbitrário da sequência.
- **Particionamento:** formação de dois subvetores (elementos $\leq$ pivô e elementos $>$ pivô).
- **Recursão:** ordenação recursiva dos subvetores utilizando o próprio algoritmo.
- **Desempenho:** caso médio similar ao **MergeSort**; pior caso com custo $O(n^2)$.
- **Espaço:** não requer alocação de memória adicional.

QuickSort: Ideia Principal

QUICKSORT(A)

```
1:  if length( $A$ ) < 2 then
2:      return  $A$ 
3:  else
4:       $p \leftarrow \text{ExtraiPivo}(A)$ 
5:       $m_1 \leftarrow \text{QuickSort}(\text{MenoresOuIguais}(A, p))$ 
6:       $m_2 \leftarrow \text{QuickSort}(\text{Maiores}(A, p))$ 
7:      return  $m_1 ++ \langle p \rangle ++ m_2$ 
```

- No pseudocódigo acima, a operação $++$ significa concatenação de vetores e $\langle p \rangle$ um vetor unitário contendo p .
- Similarmente ao **MergeSort**, implementações eficientes de **QuickSort** requerem a minimização da movimentação de dados por meio do uso de índices e reaproveitamento do espaço do próprio vetor sendo ordenado.

QuickSort: Main (Eficiente)

QUICKSORT($A, start, end$)

```
1:  if  $start < end$  then  
2:       $p \leftarrow \text{Partition}(A, start, end)$   
3:      Quicksort( $A, start, p - 1$ )  
4:      Quicksort( $A, p + 1, end$ )
```

Obs. A rotina **Partition** executa ao mesmo tempo a *extração do pivô* e o particionamento dos demais valores, devolvendo o índice p do pivô. Os elementos anteriores a p são menores que o pivô, e os posteriores a p , maiores. Para ordenar todo o vetor, a chamada inicial é **Quicksort**($A, 1, \text{length}(A)$).

QuickSort: Partition (Eficiente)

PARTITION($A, start, end$)

```
1:   $x \leftarrow A[end]$ 
2:   $i \leftarrow start - 1$ 
3:  for  $j \leftarrow start$  to  $end - 1$  do
4:      if  $A[j] \leq x$  then
5:           $i \leftarrow i + 1$ 
6:          Swap( $A, i, j$ )
7:  Swap( $A, i + 1, end$ )
8:  return  $i + 1$ 
```

- x é o pivô (aqui extraído da última posição)
- i é o índice do **último** valor menor ou igual a x
- o laço 3-6 faz j percorrer todo vetor do início ao fim (excluindo end). Se encontra um valor menor ou igual a x , o joga para o início por meio de uma troca e ajusta o índice i
- ao final, faz uma troca para posicionar o pivô após i e devolve o seu índice

QuickSort: Correção e Terminação

Tal qual no caso do **MergeSort**, utilizamos **indução** para demonstrar a correção do **QuickSort**.

Determine:

1. Argumentos para a correção do QuickSort
2. Argumentos para a terminação do QuickSort

Ordenação em Python

Implementação em Python

Python Code

↺ Start Over

▶ Run Code

```
1 import random
2 import time
3 import matplotlib.pyplot as plt
4 import sys
5 import numpy as np
6
7 # Variável para visualização do laço intermediário dos algoritmos
8 debug = False
9
10 #####
11
12 # INSERTION SORT
13
14 def insertion_sort(array):
15     n = len(array)
16     for atual in range(1, n):
17         chave = array[atual]
18         anterior = atual - 1
19         while anterior >= 0 and array[anterior] > chave:
20             array[anterior + 1] = array[anterior]
21             anterior -= 1
22         array[anterior + 1] = chave
23
24     if debug:
```

```
25         print(array, anterior, atual)
26
27
28 #####
29
30 # SELECTION SORT
31
32
33 def swap(array, idx1, idx2):
34     temp = array[idx1]
35     array[idx1] = array[idx2]
36     array[idx2] = temp
37
38
39 def selection_sort(array):
40     n = len(array)
41     for i in range(0, n):
42         min_idx = i
43         for j in range(i, n):
44             if array[j] < array[min_idx]:
45                 min_idx = j
46         swap(array, i, min_idx)
47         if debug:
48             print(array)
49
50
51 #####
52
53 # BUBBLE SORT
54
55
```

```

56 def bubble_sort(array):
57     n = len(array)
58     for i in range(0, n - 1):
59         for j in range(0, n - 1 - i):
60             if array[j] > array[j + 1]:
61                 swap(array, j, j + 1)
62         if debug:
63             print(array)
64
65
66 #####
67
68 # MERGE SORT (versão imperativa)
69
70 def merge_sort(array, aux, inicio, fim):
71     if inicio < fim:
72         meio = (inicio + fim) // 2
73         if debug:
74             print(array[inicio:fim + 1])
75             print(array[inicio:meio + 1], array[meio + 1:fim + 1])
76         merge_sort(array, aux, inicio, meio)
77         merge_sort(array, aux, meio + 1, fim)
78         if debug:
79             print(array[inicio:meio + 1], array[meio + 1:fim + 1])
80         merge(array, aux, inicio, meio, fim)
81         if debug:
82             print(array[inicio:fim + 1])
83
84
85 def merge(array, aux, inicio, meio, fim):
86     for i in range(inicio, fim + 1):

```

```
87         aux[i] = array[i]
88     esquerda = inicio
89     direita = meio + 1
90     atual = inicio
91     while esquerda <= meio and direita <= fim:
92         if aux[esquerda] < aux[direita]:
93             array[atual] = aux[esquerda]
94             esquerda += 1
95         else:
96             array[atual] = aux[direita]
97             direita += 1
98     atual += 1
99
100     while esquerda <= meio:
101         array[atual] = aux[esquerda]
102         esquerda += 1
103         atual += 1
104     while direita <= fim:
105         array[atual] = aux[direita]
106         direita += 1
107         atual += 1
108
109
110     #####
111
112     # MERGE SORT (versão funcional)
113
114
115     def merge_sort_2(array):
116         n = len(array)
117         if n < 2:
```

```

118         return array
119     else:
120         metade_esq = merge_sort_2(array[:n // 2])
121         metade_dir = merge_sort_2(array[n // 2:])
122         return merge_2(metade_esq, metade_dir)
123
124
125 def merge_2(left, right):
126     merged = []
127     i = 0
128     j = 0
129     while i < len(left) and j < len(right):
130         if left[i] < right[j]:
131             merged.append(left[i])
132             i += 1
133         else:
134             merged.append(right[j])
135             j += 1
136
137     # Adicionar os restantes
138     merged.extend(left[i:])
139     merged.extend(right[j:])
140     return merged
141
142
143 #####
144
145 # QUICKSORT (versão imperativa)
146
147
148 def quick_sort(array, inicio, fim):

```

```
149     if inicio < fim:
150         if debug:
151             print(array[inicio:fim + 1])
152         pivo_idx = partition(array, inicio, fim)
153         if debug:
154             print(array[inicio:pivo_idx], array[pivo_idx], array[pivo_idx + 1:fim + 1])
155         quick_sort(array, inicio, pivo_idx - 1)
156         quick_sort(array, pivo_idx + 1, fim)
157
158
159 def partition(array, inicio, fim):
160     pivo = array[fim]
161     menores_idx = inicio - 1
162     for atual in range(inicio, fim):
163         if array[atual] <= pivo:
164             menores_idx += 1
165             swap(array, menores_idx, atual)
166     swap(array, menores_idx + 1, fim)
167     return menores_idx + 1
168
169
170 #####
171
172 # QUICK SORT (versão funcional)
173
174
175 def quick_sort_2(array):
176     if len(array) < 2:
177         return array
178     # extrai pivô
179     pivo = array[0]
```

```

180     restante = array[1:]
181     # ordena recursivamente menores e maiores que pivô
182     menores = quick_sort_2(list(filter(lambda x: x <= pivo, restante)))
183     maiores = quick_sort_2(list(filter(lambda x: x > pivo, restante)))
184     # devolve lista ordenada
185     return menores + [pivo] + maiores
186
187
188 def benchmark(quantidade_vetores=5, executar_lentos=False):
189     # Aumentar limite de recursão
190     sys.setrecursionlimit(5000)
191
192     print("\n--- Teste de Desempenho ---")
193     debug = False
194
195     # Usaremos os valores exatos modificados pelo usuário
196     ns_rapidos = [1000, 5000, 10000, 25000, 50000]
197     ns_lentos = [1000, 2000, 5000, 10000, 25000] # O(n^2)
198
199     algoritmos_lentos = ['Insertion', 'Selection', 'Bubble'] if executar_lentos else []
200     algoritmos_rapidos = ['Merge\n(Imperat.)', 'Merge\n(Funcional)', 'Quick\n(Imperat.)',
201
202     tempos_medias = {alg: [] for alg in algoritmos_lentos + algoritmos_rapidos}
203     tempos_desvios = {alg: [] for alg in algoritmos_lentos + algoritmos_rapidos}
204     ns_executados_lentos = []
205
206     for n in ns_rapidos:
207         resultados_rodadas = {alg: [] for alg in algoritmos_lentos + algoritmos_rapidos}
208
209         # Rodar múltiplas vezes para calcular média e desvio
210         for _ in range(quantidade_vetores):

```

```
211     vetor_aleatorio = [random.randint(0, n) for _ in range(n)]
212
213     # Algoritmos O(N log N)
214
215     # Merge Sort (Imperativo)
216     V = vetor_aleatorio.copy()
217     aux_v = V.copy()
218     inicio = time.time()
219     merge_sort(V, aux_v, 0, len(V) - 1)
220     resultados_rodadas['Merge\n(Imperat.)'].append(time.time() - inicio)
221
222     # Merge Sort (Funcional)
223     V = vetor_aleatorio.copy()
224     inicio = time.time()
225     merge_sort_2(V)
226     resultados_rodadas['Merge\n(Funcional)'].append(time.time() - inicio)
227
228     # Quick Sort (Imperativo)
229     V = vetor_aleatorio.copy()
230     inicio = time.time()
231     quick_sort(V, 0, len(V) - 1)
232     resultados_rodadas['Quick\n(Imperat.)'].append(time.time() - inicio)
233
234     # Quick Sort (Funcional)
235     V = vetor_aleatorio.copy()
236     inicio = time.time()
237     quick_sort_2(V)
238     resultados_rodadas['Quick\n(Funcional)'].append(time.time() - inicio)
239
240     # Algoritmos O(N^2)
241     if n in ns_lentos and executar_lentos:
```

```

242         # Insertion Sort
243         V = vetor_aleatorio.copy()
244         inicio = time.time()
245         insertion_sort(V)
246         resultados_rodadas['Insertion'].append(time.time() - inicio)
247
248         # Selection Sort
249         V = vetor_aleatorio.copy()
250         inicio = time.time()
251         selection_sort(V)
252         resultados_rodadas['Selection'].append(time.time() - inicio)
253
254         # Bubble Sort
255         V = vetor_aleatorio.copy()
256         inicio = time.time()
257         bubble_sort(V)
258         resultados_rodadas['Bubble'].append(time.time() - inicio)
259
260     for alg in algoritmos_rapidos:
261         tempos_medias[alg].append(np.mean(resultados_rodadas[alg]))
262         tempos_desvios[alg].append(np.std(resultados_rodadas[alg]))
263
264     if n in ns_lentos and executar_lentos:
265         ns_executados_lentos.append(n)
266         for alg in algoritmos_lentos:
267             tempos_medias[alg].append(np.mean(resultados_rodadas[alg]))
268             tempos_desvios[alg].append(np.std(resultados_rodadas[alg]))
269
270     print(f"Testado N={n} (Média de {quantidade_vetores} vetores)")
271
272     # Gerando o gráfico

```

```

273 plt.figure(figsize=(10, 6))
274
275 cores = {'Insertion': 'red', 'Selection': 'orange', 'Bubble': 'gold',
276          'Merge\n(Imperat.)': 'lightgreen', 'Merge\n(Funcional)': 'green',
277          'Quick\n(Imperat.)': 'lightblue', 'Quick\n(Funcional)': 'blue'}
278
279 if executar_lentos:
280     for alg in algoritmos_lentos:
281         plt.errorbar(ns_executados_lentos, tempos_medias[alg], yerr=tempos_desvios[alg])
282
283 for alg in algoritmos_rapidos:
284     plt.errorbar(ns_rapidos, tempos_medias[alg], yerr=tempos_desvios[alg], fmt='-o', label=alg)
285
286 plt.xlabel('n do Vetor (N)')
287 plt.ylabel('Tempo Médio (segundos)')
288 plt.title(f'Tempo de Execução dos Algoritmos de Ordenação (Média de {quantidade_vetores} vetores)')
289 plt.legend()
290 plt.grid(True)
291
292 plt.tight_layout()
293 plt.show()
294
295
296 # Vetor desordenado
297 l1 = [9, 1, 0, 2, 4, 5, 3, 8, 7, 6]
298 aux = l1.copy()
299
300 #insertion_sort(l1)
301 #selection_sort(l1)
302 #bubble_sort(l1)
303 merge_sort(l1, aux, 0, len(l1) - 1)

```

```
304 #quick sort(l1, 0, len(l1) - 1)
```



Ferramentas de Verificação

Ferramentas de Verificação

A **verificação formal de software** utiliza invariantes de laço, pré/pós-condições e Lógica de Hoare para provar matematicamente que programas atendem a suas especificações:

- **Dafny**: Linguagem desenvolvida pela *Microsoft Research* voltada para verificação formal. Exige anotações de invariantes (`invariant`) e pré/pós-condições (`requires/ensures`), provando a corretude de algoritmos automaticamente via Z3.
- **Frama-C / ACSL**: Plataforma para análise e verificação formal de programas C. Utiliza anotações ACSL (`loop invariant`) para comprovação matemática de propriedades e ausência de *bugs*.
- **SPARK / Ada**: Conjunto de ferramentas de verificação para Ada, utilizado em sistemas críticos (aeroespacial, ferroviário e médico) para provar matematicamente a corretude usando invariantes.
- **KeY**: Sistema de verificação dedutiva para Java que provê verificação baseada em invariantes especificados em JML (*Java Modeling Language*).
- **CBMC**: *Bounded Model Checker* para C/C++ que analisa e verifica automaticamente invariantes de laço, asserções e segurança de memória.

